

Ingeniería del Software II

Práctica #6 – Interpretación Abstracta

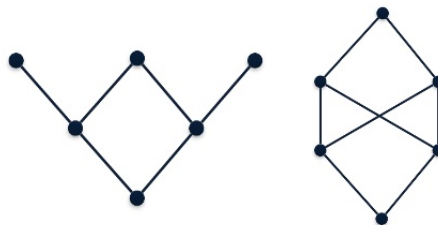
Cuando se pide un reticulado, si el mismo es finito, mostrarlo mediante un diagrama de Hasse. Sino indicar el dominio, la relación de orden $\sqsubseteq$, el supremo e ínfimo.

Ejercicio 1

1. Definir un reticulado usando el orden parcial $\langle \mathbb{Z}, \leq \rangle$. ¿Existen elementos $\top$ y $\perp$?
2. Definir un reticulado completo sobre el conjunto $\mathbb{Z} \cup \{-\infty, \infty\}$.

Ejercicio 2

Sean los órdenes parciales dado por los siguientes diagramas de Hasse:



1. Indicar si son reticulados.
2. En caso que no lo sean, indicar por qué no lo son y cómo los repararían.

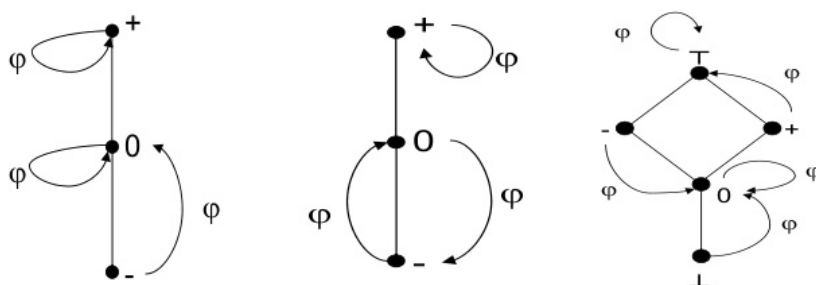
Ejercicio 3

Sea $S = \{A', B', C'\}$

1. Armar un reticulado de altura 2 donde los elementos de S no sean comparables.
2. Armar el reticulado de partes de S . ¿Qué altura tiene?
3. Sea $N = \{1, 2\}$, donde $1 \sqsubseteq 2$. Armar el reticulado de pares $S \times N$. ¿Qué pasaría si los elementos de S fueran comparables y fueran un orden total?

Ejercicio 4

Sean las siguientes funciones $\varphi : S \mapsto S$, donde S está dado por el diagrama de Hasse y φ por las flechas conectando nodos.



1. Calcular $\varphi(\sqcup X)$ para todo $X \subseteq S$
2. Enumerar todas las cadenas incrementales (*increasing chains*) en S
3. ¿Es φ monótona?
4. Computar iterativamente el lfp (mínimo punto fijo) de φ .

Ejercicio 5

¿El teorema del punto fijo también se cumple sin la suposición de que f es monótona? Si es así, justificar con una prueba; si no, proveer un contraejemplo.

Ejercicio 6

Sea el reticulado de $Sign = Flat(\{-, 0, +\})$:

1. Extenderlo para incluir 2 símbolos que representen los valores mayores iguales a cero y los menores que cero.
2. Definir la operación suma en el nuevo reticulado.

Ejercicio 7

Sea D el conjunto que representa funciones de variables a signos. $D : Var \mapsto Sign$. Es decir cada elemento $d \in D$ es una función.

1. Definir la operación $\sqsubseteq$ que determine que dados $d1, d2 \in D$ $d1 \sqsubseteq d2$ si $d1$ es igual o más preciso que $d2$.
2. Definir la operación $\sqcup$ que represente el supremo en D .
3. Usaremos la notación $d[x \mapsto n]$ para denotar una nueva función d' donde todos los elementos distintos a x están mapeados al mismo valor, salvo $d'(x) = n$.

Extendemos el lenguaje con la instrucción $x+ = E$. Dado un elemento $d \in D$ esta operación modifica el estado de la variable x sumándole el valor del signo de E . Formalmente: $\llbracket x+ = E \rrbracket(d) = d[x \mapsto x + sign(E)]$, donde $sign(E)$ es el valor de E en el reticulado $Sign$.

Es $\llbracket x+ = E \rrbracket$ monótona? Justificar.

Ejercicio 8

Supongamos que tenemos programas que manipulan datos sensibles. Estos datos se encuentran en variables.

Usando la función `sensible(x)` decimos que el dato en la variable x es sensible y con la función `insensible(x)` decimos que x no es sensible. Un dato sensible puede pasarse a otra variable si es leído. Por ejemplo $y = x + 1$ hace que y sea sensible porque leyó un dato sensible que venía de x (si x es sensible).

1. Definir un reticulado que modele el uso de variables sensibles e insensibles.
2. Definir un análisis Dataflow para calcular para cada punto del programa si una variable puede ser sensible o no. Es decir para cada nodo n del CFG, $OUT[n](x)$ debería indicar si x es sensible o no.
3. Calcular el resultado del análisis dataflow para el siguiente programa.

```

1  x = 1;
2  sensible(x);
3  y = input;
4  if(y>0)
5      z = x + 1;
6  else
7      z = 0
8  insensible(x);

```

Ejercicio 9

Sea el siguiente programa:

```

1  y = 0;
2  x = 0;
3  z = 1;
4  while (true) {
5      x = y + 1;
6      if (y==0)
7          x = 0;
8  }

```

1. Construir el CFG del programa.
2. Calcular las igualdades de la forma $x = y$.
 - Definir un reticulado para trackear igualdad entre variables (hint: usar conjunto de pares de variables)
 - Definir la funciones de transferencia para las instrucciones que aparecen en el programa.
 - Definir las ecuaciones de Dataflow para el programa.
 - Calcular la solución a las ecuaciones (usando cualquier algoritmo de punto fijo)

Ejercicio 10

Sobre el mismo programa del ejercicio anterior. Computar el signo de todas las variables (definiendo el análisis de dataflow correspondiente)

Ejercicio 11

Se quiere analizar el estado de un buffer de texto con las siguientes operaciones:

- **crearBuffer()**: Crea un nuevo buffer vacío
- **agregarCaracter(b, c)**: Agrega un caracter al buffer
- **vaciarBuffer(b)**: Vacía el buffer

Los estados posibles son: Buffer no creado, vacío (creado sin contenido), un único carácter, un texto (más de un carácter)

- a. Definir un reticulado B para modelar los estados del buffer. Dibujar el diagrama de Hasse.
- b. Definir la semántica abstracta de las operaciones completando la siguiente tabla:

Estado (b)	agregarCaracter(b,c)	vaciarBuffer(b)
$\perp$	?	?

Ejercicio 12

Se busca realizar un análisis de dataflow para predecir el estado de una variable de tipo `Queue` (cola), que representa colas no acotadas.

Se extiende el lenguaje TIP para incluir las expresiones: `crearQueue()` que crea una cola vacía, `enqueue(q, e)` que devuelve la cola `q` con el elemento `e` añadido al final, y `dequeue(q)` que devuelve la cola `q` sin su primer elemento (requiere que `q` no esté vacía).

Notar que `enqueue(q, e)` y `dequeue(q)` requieren que la cola `q` exista (es decir, que haya sido creada previamente con `crearQueue()`).

Por ejemplo en el siguiente programa:

```
void main() {
  var q, q1, q2, q3, q4;
  q = crearQueue();    // q esta vacía
  q1 = enqueue(q, 5);  // q1 tiene un elemento
  q2 = enqueue(q1, 8); // q2 tiene dos elementos
  q3 = dequeue(q2);    // q3 tiene un elemento
  q4 = dequeue(q3);    // q4 esta vacía
}
```

- Se desea determinar si una cola está **no inicializada**, está **vacía**, **tiene exactamente 1 elemento** o **tiene más de 1 elemento**. Dibuja el reticulado Q que modele estos estados, indicando quien es el elemento Top ($\top$) y Bottom ($\perp$).
- Definir la semántica abstracta para las operaciones `enqueue` y `dequeue`. Tu definición debe modelar de la forma más precisa posible, pero segura (sobre-aproximada), el estado de la cola. Completa la siguiente tabla para cada elemento del reticulado que definiste en el punto anterior.

Estado Actual (q)	enqueue(q, e)	dequeue(q)
...	...	...
$\top$	...	...
$\perp$	...	...